SeldonH — Mémo technique

Fonctionnement général de l'application

Philippe Dufresne

4 novembre 2026

Résumé

SeldonH est une application Qt/C++ de dialogue avec un modèle de langage (local par llama.cpp, ou distant par une API compatible OpenAI) destinée aux archéologues et géologues. Elle réalise des datations simples, calcule des Events au sens de ChronoModel, produit des fichiers de modèle `.chr` et aide à interpréter les résultats. Le principe directeur est de *séparer le langage du calcul* : le modèle de langage comprend la question, choisit et commente ; tous les calculs (calibrations, MCMC, fichiers) sont faits par du code C++ déterministe, porté de ChronoModel, sur l'ordinateur de l'utilisateur. Ce mémo décrit l'architecture (graphe fonctionnel), l'algorithme général de traitement d'une question et les principaux modules. Mise à jour du 4 novembre 2026 : mode calculatrice, calcul unique `relation` (termes de provenances mélangées, corrélation, formules), tableaux de datations, serveur MCP, test automatique de l'appel d'outils, réglages CPU.

Table des matières

1 Principes de conception **2**

2 Graphe fonctionnel **2**

3 Algorithme général **4**

4 Sélection des compétences **6**

5 Moteurs de langage **6**
5.1 Modèle local (llama.cpp) 6
5.2 Modèle distant 7

6 Calculs de datation **7**
6.1 Dates isolées 7
6.2 Calcul d'un Event (MCMC) 7
6.3 Relations entre dates : un seul calcul `relation` 7
6.4 Tableaux de datations 9
6.5 Autres calculs 9

7 Création d'un modèle ChronoModel **9**

8 Interprétation des résultats **10**

9 Serveur MCP `seldonh-mcp` **10**

10 Stockage, déploiement et tests **10**

11 Limites connues **10**

1 Principes de conception

1. **Le modèle ne calcule jamais.** Un petit modèle ne sait pas calibrer et invente des règles. Les méthodes de datation sont donc reprises de ChronoModel en C++ (`src/methods/`) et exécutées localement. Le modèle reçoit le *texte* des résultats exacts et les commente.

2. **Le modèle ne produit jamais le `.chr` complet.** Il écrit une spécification JSON courte (une dizaine de champs) ; `ChrBuilder` ajoute les quelque 200 champs obligatoires (identifiants, UUID, réglages MCMC, plage d'étude, positions…) et contrôle la cohérence.

3. **Connaissances par fiches.** Le savoir-faire est rangé dans des *compétences* (fiches Markdown) choisies à chaque question selon ses mots-clés, dans un budget de texte. On enrichit l'application en ajoutant des fiches, sans recompiler.

4. **Un seul jeu de calculs pour trois transports.** Modèle local (bloc `"‘calcul`), modèle distant (appel d'outils natif) et autre assistant (serveur MCP `seldonh-mcp`) déclenchent les mêmes fonctions (`src/methods/common/Tools.*`) ; l'historique est identique, on peut changer de moteur en cours de conversation.

5. **La fiche de SeldonH est la seule source des chiffres.** Quand SeldonH a fait le calcul seul et que la question ne demande pas d'explication, le modèle n'est pas relancé (*mode calculatrice*) ; sinon il reçoit les valeurs « déjà affichées » avec la consigne de ne pas les recopier.

6. **Le modèle sélectionne, SeldonH manipule les valeurs.** Pour un tableau de datations, le modèle ne reçoit qu'un résumé et demande des sélections avec une action ; il ne recopie jamais une valeur.

7. **Confidentialité.** Avec le modèle local, rien ne quitte la machine. Avec un modèle distant, seuls la conversation, les fiches choisies, les pièces jointes et les paramètres d'appel d'outils sont envoyés ; les courbes de calibration et les calculs restent locaux. La clé API est rangée dans le trousseau du système (QtKeychain).

2 Graphe fonctionnel

La figure 1 montre les modules et les flux. Les flèches pleines sont les flux de données d'une question ; les flèches en tirets sont les boucles (relance du modèle après un calcul, retour des résultats de ChronoModel).

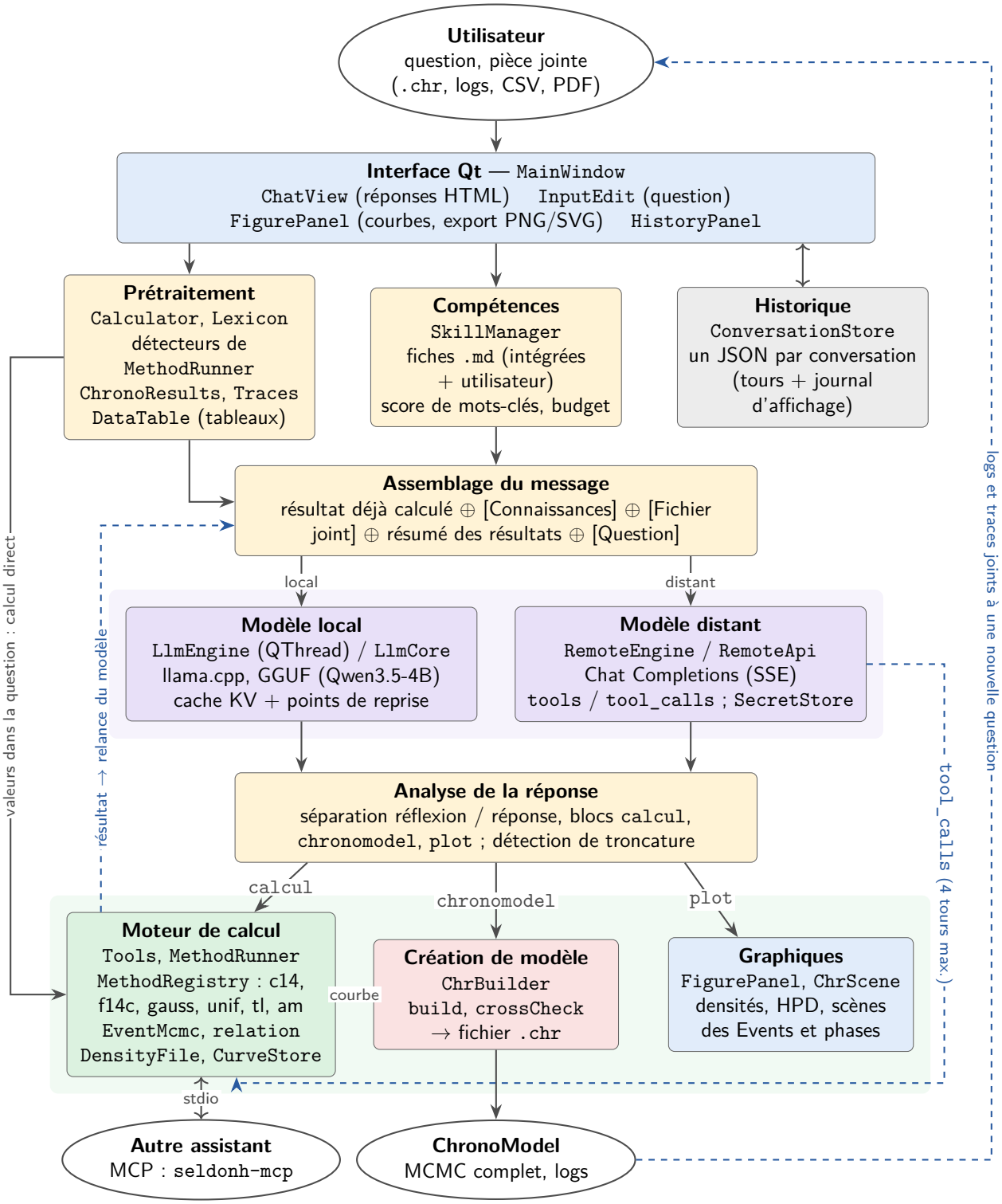


Figure 1: Graphe fonctionnel de SeldonH. La réponse est affichée en flux dans l'interface pendant sa génération. Orange : préparation et analyse du texte ; violet : moteurs de langage ; vert : calculs locaux déterministes ; rouge : création du modèle ChronoModel ; gris : stockage.

Rôle des modules

Module	Rôle
`MainWindow`	Orchestration d'une question (algorithme 1), gestion des tours, mode calculatrice, liens `seldon://` (enregistrer un `.chr`, un fichier de densité, un extrait de tableau, demander un commentaire, déplier le raisonnement, rappeler une figure).
`ChatView`, `MathText`	Rendu Markdown → HTML, formules LaTeX du modèle → HTML.
`FigurePanel`, `ChrScene`	Courbes (densités, HPD remplie, superposition ou une densité par ligne), nuages de points (corrélation), export PNG/SVG, scènes des Events et des phases d'un `.chr`.
`Calculator`	Calculatrice locale (« 2+3 », `ADtoBP(-550)`) sans appel au modèle.
`Lexicon`	Ramène une question en espagnol, italien ou allemand aux mots-clés français.
`SkillManager`	Lecture des fiches (en-tête : `keywords`, `priority`, `always`) et sélection.
`LlmCore` / `LlmEngine`	Enveloppe C++ de llama.cpp (sans Qt) ; adaptateur Qt dans un `QThread`.
`RemoteEngine` / `RemoteApi`	API Chat Completions compatible OpenAI, flux SSE, appel d'outils.
`ToolsProbe`	Test automatique de l'appel d'outils d'un serveur distant (natif, en texte, ignoré, refusé), mémorisé par serveur et modèle pendant 30 jours.
`CpuTopology`	Nombre de threads « automatique » : cœurs physiques performance (génération) et tous les cœurs physiques (prompt), sous macOS, Linux et Windows.
`AutoTune`	« Optimiser pour ce modèle » : contexte, longueur de réponse, budget des fiches.
`MethodRunner`, `Tools`	Détecteurs de datations et de relations, normalisation et validation des requêtes, exécution (dont `relation`, §6.3), fiche de résultat, texte pour le modèle ; définition des 14 outils.
`MethodRegistry`	Une classe `Method` par technique (c14, f14c, gauss, unif, tl, am).
`EventMcmc`	Estimation bayésienne d'un Event (modèle EDM2), diagnostics $\hat{R}$ et ESS.
`CurveStore`, `RefCurve`	Courbes de référence : dossier utilisateur, `Calib` de ChronoModel, courbes intégrées.
`ChrBuilder`	Spécification JSON → `.chr` valide ; contraintes en toutes lettres ; vérification croisée avec le texte de l'utilisateur.
`ChronoResults`	Lecture des logs et résultats de ChronoModel joints, résumé dans un budget.
`Traces`	Chaînes MCMC exportées par ChronoModel (`Chain_all_Events.csv`, `Chain_all_Phases.csv`…), gardées tirage par tirage ; noms de colonnes en clair (« début de la phase A » → `Phase A Begin`).
`DataTable`, `Spreadsheet`	Tableaux de datations (`.xlsx`, `.ods`, CSV, collés) : rôles des colonnes, méthode de chaque ligne, sélections, actions (§6.4).
`McpServer`	Serveur MCP `seldonh-mcp` (QtCore seul) : les 14 outils pour un autre assistant.
`ConversationStore`	Historique (écriture atomique `QSaveFile`).

3 Algorithme général

L'algorithme 1 décrit le traitement d'une question, de la saisie à l'enregistrement de la conversation. Les étapes 1 à 4 sont des *courts-circuits* : elles répondent sans le modèle de langage.

Remarques.

- Un calcul est fait *au plus une fois* par question : un tour marqué « déjà calculé » (`followUp`) ne relance pas de calcul, ce qui évite les boucles avec le modèle local.
- Le modèle local n'a qu'un tour de suite après un bloc `calcul` ; le modèle distant peut enchaîner jusqu'à 4 appels d'outils, puis la requête est envoyée avec `tool_choice: none`.
- Si le serveur distant refuse le champ `tools`, la requête est rejouée sans outils et le bloc `calcul` prend le relais. Un appel d'outil est réécrit en bloc `calcul` dans l'historique.

Algorithme 1 Traitement d'une question (`MainWindow::sendMessage` et suite)

Entrées : question q, pièce jointe éventuelle a, conversation C (tours précédents), réglages (format de date, budget B des fiches, moteur).
Sorties : réponse affichée, figures, éventuel fichier `.chr`, conversation enregistrée.

si $q =$ `/aide` **alors** afficher le mode d'emploi ; **fin**
si `Calculator.isCalculation`(q) **alors** afficher le résultat exact ; **fin** *rien n'est envoyé au modèle*
si q demande un dessin **et** un `.chr` est connu **alors** dessiner les scènes ;
  si la question ne demande que le dessin **alors fin**
si q ou a contient un tableau de datations **alors** le lire comme tableau courant, afficher sa fiche ;
  si il n'y a pas d'autre consigne **alors fin**
$D \leftarrow$ `recentDatings`(C) *datations des 8 dernières questions*
si `requestsFromQuestion`$(q) \neq \emptyset$ **ou** `relationWithPrevious`$(q, D) \neq \emptyset$
  et `asksCommentary`(q) est faux **alors** calculer, afficher la fiche et le lien « Demander un commentaire »; **fin**
mode calculatrice : aucune génération, marche sans modèle chargé
si aucun modèle de langage n'est prêt **alors** message d'état ; **fin**
$q_k \leftarrow$ `Lexicon.translate`(q) *mots-clés en français*
$m \leftarrow$ `isModelRequest`(q_k) ; $M \leftarrow$ `datingMethodsIn`(q_k)
$P \leftarrow$ fiches imposées : 03 si m ; 13 si âge-profondeur ; fiche de chaque méthode de M ;
  19 (phases, Events) ou 11 (logs) si des résultats ChronoModel sont présents ; 21 si une relation est demandée ;
  22 et résumé du tableau courant s'il y en a un
$K \leftarrow$ `SkillManager.select`$(q_k,\ m\ ?\ 0 : B,$ exclues, $P)$ *voir §4*
$R \leftarrow$ `MethodRunner.analyzeQuestion`(q) *valeurs écrites dans la question*
si non R.trouvé **alors** $R \leftarrow$ recalcul de la datation précédente (autre niveau, autre période)
si R.trouvé **alors** afficher la fiche (HPD, médiane, mode) et la densité
$u \leftarrow R$.texte $\oplus\ K \oplus\ a \oplus$ résumé des résultats $\oplus\ q$ *message vu par le modèle*
ajouter le tour (q, u) à C ; enregistrer C
répéter *boucle agent*
  générer la réponse en flux (local : gabarit + cache KV ; distant : SSE avec déclaration des outils)
  tant que le moteur distant renvoie des `tool_calls` **et** tours ≤ 4 **faire**
    exécuter chaque outil localement (`Tools`) ; réinjecter le texte du résultat ; poursuivre
  afficher la réponse : réflexion séparée, blocs `calcul`/`chronomodel` masqués, `plot` $\rightarrow$ panneau
  si bloc `chronomodel` **alors**
    $S \leftarrow$ spécification ; `ChrBuilder.build`(S) ; `crossCheck`(S, q)
    afficher le résumé, les avertissements et le lien « Enregistrer le modèle (`.chr`) »
  si bloc `calcul` **et** le tour n'a pas déjà été calculé **alors**
    $R' \leftarrow$ `MethodRunner.run`(blocs) ; afficher fiche et figures
    si la question ne demande pas d'explication **alors** sortir *mode calculatrice*
    ajouter un tour synthétique $u \leftarrow R'$.texte ; **continuer** *le modèle commente*
  sinon sortir de la boucle
si la réponse est coupée (limite de longueur, contexte plein, flux interrompu) **alors**
  proposer « Continuer la réponse » (la suite s'ajoute au même tour)
enregistrer C (tours + journal d'affichage)

- Les fiches des tours passés ne sont pas renvoyées : seul le tour courant les reçoit, ce qui garde le prompt court.
- Un tour en mode calculatrice (`calcOnly`) n'a pas de réponse du modèle : il n'est pas envoyé comme paire question / réponse, mais placé en contexte en tête du message suivant (`calcOnlyContext`), faute de quoi un petit modèle recopiait le texte de remplacement. L'export du prompt montre exactement ce qui est envoyé.
- Préférence « Commentaire après un calcul » : si la question le demande (défaut), jamais, toujours.

4 Sélection des compétences

Chaque fiche `.md` porte un en-tête (`name`, `description`, `keywords`, `priority`, `always`). La fiche d'identité (`always: true`) va dans le message système ; les autres sont placées devant la question. Le score d'une fiche pour une question q (sans accents, en minuscules) est

$$s = 10\Big(\sum_{k\in\text{mots-clés}} w(k) + \sum_{w\in\text{description}} [\,w \in q\,]\Big) + \text{priorité},$$

$$w(k) = \begin{cases} 4 & k \text{ expression trouvée dans } q \\ 3 & k \text{ mot de } q \\ 2 & |k| > 4 \text{ et } k \text{ sous-chaîne de } q \text{ (pluriel, préfixe)}, \\ 0 & \text{sinon.} \end{cases}$$

Les fiches imposées passent d'abord, puis les fiches par score décroissant tant que le budget (6000 caractères par défaut) n'est pas dépassé. Pour une création de modèle, le budget est mis à zéro : seules la fiche commune (03) et les fiches des méthodes citées sont fournies (exemple réel : 6 100 caractères au lieu de 18 500). Une fiche utilisateur de même nom remplace la fiche intégrée.

5 Moteurs de langage

5.1 Modèle local (llama.cpp)

`LlmCore` formate la conversation avec le gabarit du modèle, n'évalue que le texte nouveau (cache KV), génère en flux et oublie les plus anciens échanges si le contexte est plein. Modèle recommandé : Qwen3.5-4B en Q4_K_M (≈ 2,7 Go, Apache 2.0), réflexion désactivée par défaut (préfixe `<think></think>` vide) ; le bouton « Réfléchir » l'active, grisé si le gabarit n'a pas `enable_thinking`. Le raisonnement peut être affiché (replié sous la réponse) mais n'est jamais renvoyé au modèle.

Performances CPU. llama.cpp b11794. Sur macOS, une compilation par architecture puis `lipo` : le build universel en un seul passage tombait en `GGML_CPU_GENERIC` (ni NEON optimisé, ni AVX2) ; gain mesuré ×2,2 (6,8 → 15,7 tok/s, i9-8950HK, Qwen3.5-2B). Threads « automatique » : cœurs physiques performance pour la génération, tous les cœurs physiques pour le prompt (`CpuTopology`) ; au-delà des cœurs libres le débit s'effondre (attente active de ggml) quand un MCMC ou l'interface tourne en même temps.

Modèles hybrides. Qwen3.5 mêle couches récurrentes (Gated DeltaNet) et couches d'attention ; l'état récurrent ne peut pas être raccourci. `LlmCore` sauve donc au plus trois *points de reprise* de l'état récurrent (≈ 50 Mo chacun) : fin du message système, début et fin du dernier message utilisateur. Au tour suivant, on restaure le point le plus avancé contenu dans le préfixe commun, puis on raccourcit les couches d'attention.

5.2 Modèle distant

`RemoteEngine` parle à toute API Chat Completions compatible OpenAI (Base URL + clé). Réponse en flux SSE ; troncature détectée par `finish_reason: "length"` ou par l'absence de `[DONE]`. Les 14 outils (`calibrer_radiocarbone`, `calibrer_f14c`, `dater_luminescence`, `densite_gauss`, `densite_uniforme`, `dater_archeomagnetisme`, `calculer_evenement`, `combiner_dates`, `comparer_dates`, `analyser_traces`, `calibration_inverse`, `creer_densite`, `courbes_disponibles`, `tableau`) sont déclarés avec leur schéma ; leurs arguments sont validés et normalisés (virgule décimale, nombres écrits en texte) et une valeur manquante renvoie une consigne au modèle sans lancer de calcul. En mode calculatrice, les résultats ne sont pas renvoyés au serveur (sauf si un outil n'a pas pu s'exécuter).

Appel d'outils : Automatique / Toujours / Jamais. En automatique, `ToolsProbe` envoie une requête factice (outil `verifier_outil`, température 0) et classe la réponse : natif (`tool_calls`), appel écrit dans le texte, ignoré, refusé, ou inconnu (réseau, 401, 429 : rien n'est conclu). Le résultat est mémorisé par serveur et modèle pendant 30 jours ; un refus vu en conversation aussi.

6 Calculs de datation

6.1 Dates isolées

Chaque méthode (`src/methods/<id>/`) reprend la vraisemblance du plugin ChronoModel correspondant ; la densité est calculée sur la grille de la courbe de référence avec un a priori uniforme, normalisée par la méthode des trapèzes, tronquée à 10^{-5} ; les régions HPD (95 % et 68 % par défaut) sont obtenues par dichotomie sur le seuil de densité. La fiche affiche HPD, médiane, mode, et la densité est tracée dans le panneau de droite. Les décimales affichées suivent l'écart-type (deux chiffres significatifs) ou un réglage fixe.

6.2 Calcul d'un Event (MCMC)

`EventMcmc` porte le moteur de ChronoModel 4.0 pour un Event sans contrainte, modèle EDM2 :

$$y_i = t_i + \delta_i, \qquad t_i \sim \mathcal{N}(\theta, \sigma_i^2), \qquad \theta \sim \mathcal{U}[t_{\min}, t_{\max}], \qquad \sigma_i^2 \mid S_0^2 \sim \text{shrinkage}(S_0^2), \qquad S_0^2 \sim \text{Inv-Gamma}(1, \beta).$$

Algorithme 2 Échantillonnage d'un Event (`EventMcmc`, d'après `MCMCLoopChrono`)

pour chaque chaîne $c = 1..N_c$ (fil d'exécution séparé, graine propre) **faire**
  initialiser t_i, σ_i, θ (`initialize_time`)
  chauffe : `num_burn` itérations
  adaptation : par lots, jusqu'à ce que tous les taux d'acceptation soient dans leur fenêtre
    (42–46 % ; 30–40 % pour t_i, σ_i d'un Event à plusieurs dates), Robbins–Monro
  acquisition : `num_iter` itérations, éclaircissement `thinning`
    pour chaque date i : bloc $(t_i, \sigma_i, \delta_i)$ par Metropolis–Hastings (calibration + shrinkage)
    décalage rigide $\theta + t_i$; Gibbs de $\theta : \mathcal{N}(\mu, 1/P)$ tronquée à la période d'étude
    marche adaptative sur $\log_{10} S_0^2$; recuit périodique ($T = 2^e$, e décroissant)
densités a posteriori de θ, t_i : noyau gaussien, largeur de bande de Sheather–Jones
diagnostics : split-$\hat{R}$ rang-normalisé (bulk et tail), ESS bulk et tail
verdict : « non convergé » si $\hat{R} \geq 1{,}05$ ou ESS $< 100N_c$; « satisfaisant » si $\hat{R} < 1{,}01$ et ESS $\geq 500N_c$

6.3 Relations entre dates : un seul calcul `relation`

Une seule demande,

```
{"method":"relation", "op":..., "from":A, "to":B, "of":C}
```

(outils `comparer_dates` et `analyser_traces`), avec six opérations : `value` (densité de C), `duration` ($B-A$ et $P(B$ postérieur à $A)$), `between` (date uniforme entre A et B), `order` ($P(A<B)$ et écart), `inside` ($P(C \in [A;B])$, avant, après) et `correlation` (matrice de corrélation et nuages de points de 2 à 8 termes, liste `terms`). Chaque terme peut être :

- une **datation indépendante** : c14, f14c, tl, gauss, unif, am, `density` (fichier du plugin Density, lu dans `<racine>/Density`), `combine` ou `event` calculés par SeldonH, avec wiggle et période d'étude ; une datation écrite en texte (« 2450 ± 30 BP », « TL 1700 ± 50 mesurée en 2025 ») est relue comme datation ;
- une **colonne des traces chargées**, éventuellement prise dans plusieurs exports du *même* calcul, par exemple `Chain_all_Events.csv` et `Chain_all_Phases.csv` (même nombre de tirages et mêmes chaînes, sinon refus) ;
- une **année fixe** (« -500 », « 500 av. J.-C. »).

Calcul exact quand il n'y a aucune colonne (sauf `between` et `correlation`) ; **tirage par tirage** dès qu'il y a une colonne. Les colonnes sont prises telles quelles : la dépendance due aux contraintes et aux phases est gardée, y compris entre deux fichiers d'export. Un mélange datation + traces porte un avertissement : la datation est supposée indépendante du modèle ChronoModel (sinon elle compte deux fois) et un Event de SeldonH n'a ni contraintes ni phases.

Formules par opération. Notations : f_X et F_X densité et fonction de répartition d'une datation indépendante (grille régulière, intégrales par trapèzes) ; a_i, b_i, c_i ($i = 1..n$) valeurs du tirage i (colonne de traces, datation tirée ou année répétée) ; $\mathbf{1}[\cdot]$ indicatrice ; y année fixe.

Opération	Calcul exact (datations indépendantes)	Tirage par tirage (traces)
`order`	$D = B - A$, $f_D(d) = \int f_A(a)\, f_B(a+d)\, da$ (grille au plus petit pas) ; $P(A<B) = 1 - F_D(0)$. Année fixe : $f_D(d) = f_B(y+d)$ ou $f_A(y-d)$.	$P(A<B) = \frac{1}{n}\sum_i \mathbf{1}[a_i < b_i]$; ex aequo comptés à part.
`duration`	densité f_D ci-contre : HPD, mode, moyenne, quartiles ; $P(B>A) = 1 - F_D(0)$.	$d_i = b_i - a_i$, densité de d par noyau ; $P(B>A) = \frac{1}{n}\sum_i \mathbf{1}[b_i > a_i]$.
`between`	toujours par tirages (10^5 sans traces) : $t = \min(a_i, b_i) + U\,\lvert b_i - a_i\rvert$, $U \sim \mathcal{U}[0;1]$, 1 à 10 valeurs par tirage ; densité de t par noyau.	
`inside`	$P(C \in [A;B]) = \int f_C\,[F_A(1-F_B) + F_B(1-F_A)]\, dc$; $P(\text{avant}) = \int f_C (1-F_A)(1-F_B)\, dc$; $P(\text{après}) = \int f_C\, F_A F_B\, dc$; borne fixe : $F(c) = \mathbf{1}[c \geq y]$.	$l_i = \min(a_i, b_i)$, $h_i = \max(a_i, b_i)$; $P(\text{dans}) = \frac{1}{n}\sum_i \mathbf{1}[l_i \leq c_i \leq h_i]$, avant : $c_i < l_i$, après : $c_i > h_i$.
`value`	f_C elle-même.	densité des c_i par noyau.
`correlation`	toujours par tirages communs (10^5 sans traces). Pearson $r = \sum_i (x_i - \bar{x})(y_i - \bar{y}) / \sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (y_i - \bar{y})^2}$; Spearman $\rho = r$ des rangs (rang moyen des ex aequo) ; intervalle approché à 95 % : $\tanh\big(\operatorname{atanh} r \pm 1{,}96/\sqrt{n_{\text{eff}} - 3}\big)$, n_{eff} = plus petit ESS bulk des deux colonnes (n pour une datation tirée).	

Datation tirée : $t_k = F^{-1}\big((k + U_k)/n\big)$, $k = 0..n-1$ (strates affectées dans un ordre aléatoire, graine fixe 12345) : indépendante des colonnes par construction, donc de corrélation nulle avec elles. **Lissage** : noyau gaussien, largeur de Sheather–Jones (Silverman en repli) ; HPD au niveau $p = \{t : f(t) \geq s\}$ de masse p (s par dichotomie). **Précision** : erreur Monte-Carlo d'une probabilité $\sqrt{p(1-p)/\text{ESS}}$ (ESS de l'indicatrice sur les chaînes) ; $\hat{R}$ et ESS bulk / tail de la quantité calculée (durée, densité d'une colonne).

Corrélation (04/11/2026). La fiche donne la matrice (r au-dessus de la diagonale avec son intervalle, ρ au-dessous) ; l'op `correlation` ajoute un nuage de points par paire (3 au plus, 2 500 tirages au plus, droite $B = A$ en tirets ; axes inversés en format d'âge), tracé par `FigurePanel` (séries `"points"` et `"dash"`, `"yreverse"`). Les autres opérations tirage par tirage ajoutent la matrice sans graphique. Détection dans la question : « corrélation », « covariance », « nuage de points », « varient ensemble ». Test (`relationMixedSources`) : $r(\text{Four}, \text{Begin}) = 0{,}89$ (théorique $20/\sqrt{500} = 0{,}894$), $r(\text{Begin}, \text{End}) = 0{,}80$, $r(\text{Four}, \text{End}) = 0{,}71$ (0,716), $r \approx 0$ avec une datation TL indépendante.

Vérification du 4 novembre 2026 (test `relationMixedSources`, deux exports synthétiques d'un même calcul). Date TL → début de phase : durée $\mathcal{N}(-25; 54)$, $P = 32$ % attendu et obtenu ; trace d'un Event → fin de phase lue dans l'autre fichier : écart-type 18 (33,5 si les colonnes étaient supposées indépendantes) ; TL, 14C, F14C, AM, uniforme, combinaison, Event de SeldonH et densité tabulée face à des traces ; outils, questions directes et refus d'exports différents. Corrections faites à cette occasion : la datation `density` n'était pas calculable (annoncée par les fiches et les outils) ; une datation sans nom mêlée à des traces s'appelait « A », « B » ou « C » (confusion avec « Phase A ») et porte maintenant le nom de sa mesure (« 14C 1650 ± 30 BP ») ; l'en-tête de la fiche nomme tous les fichiers de traces utilisés.

Datations des questions précédentes. « Combien de temps sépare la date précédente de… » : `relationWithPrevious` reprend la ou les dernières datations de la conversation (il faut un renvoi et une relation reconnue) ; pour une durée, la plus ancienne passe en premier.

6.4 Tableaux de datations

`DataTable` lit un tableau (`.xlsx` et `.ods` par `QZipReader` + XML, CSV, TSV, Markdown ou texte collé), cherche l'en-tête, reconnaît les rôles des colonnes en français et en anglais et déduit la méthode de chaque ligne. Le tableau devient le « tableau courant » de la conversation ; le modèle n'en reçoit qu'un résumé et demande `{"method":"table",...}` avec une sélection (méthodes, lignes, noms, conditions `where`) et une action : `extract` (CSV), `date` (récapitulatif), `event`, `combine`, `model` (`.chr` par ligne ou par `group_by`). Une erreur du modèle ne peut donc donner qu'une mauvaise sélection, visible dans la fiche, jamais une mauvaise valeur.

6.5 Autres calculs

- `DensityFile` : fichier de densité pour le plugin Density de ChronoModel (lois normale, log-normale, uniforme, triangulaire, trapézoïdale, exponentielle, gamma, bêta ; placement $t = \text{offset} \pm x$).
- `Combination`, `InverseCalibration` : combinaison de datations (produit des vraisemblances, test de Ward et Wilson) et calibration inverse.
- `CurveStore` : courbes cherchées dans le dossier utilisateur, puis dans `Calib` de ChronoModel, puis dans les courbes intégrées (`.14c`, `.ref`).

7 Création d'un modèle ChronoModel

Le modèle de langage écrit un bloc `"‘chronomodel` : Events, dates (`type` : `gauss`, `unif`, `tl`, `c14`, `f14c`, `am`, `density`, avec `wiggle` facultatif), bornes, phases et contraintes. `ChrBuilder` :

1. refuse les erreurs bloquantes (borne écrite comme une date, ancien nom de champ, TL sans `ref_year`) ;
2. traduit les contraintes écrites en toutes lettres (« US 12 sous US 10 », « coupe », « scellé par »…) en couples (`bwd_id` = plus ancien, `fwd_id` = plus récent), détecte contradictions et cycles ;

3. calcule la plage d'étude (HPD 95 % calibrée pour 14C et F14C, plus une marge) ;

4. complète les valeurs relevées dans des projets ChronoModel 4.0 réels (échantillonneurs, réglages MCMC, couleurs, positions) ;

5. compare la spécification au texte de l'utilisateur (`crossCheck`) : valeurs absentes du texte, références inventées, mesures du texte oubliées.

Le résultat s'affiche comme un résumé en clair et un lien d'enregistrement ; le JSON n'est montré que sur demande.

8 Interprétation des résultats

Les logs de ChronoModel joints (`Log_Model_Description`, `Log_MCMC_Initialization`, `Log_MCMC_Adaptation`, `Log_Posterior_Distrib_Stats`) sont lus par `ChronoResults`, résumés dans un budget adapté au contexte du modèle, et accompagnés de la fiche de diagnostic. La démarche imposée au modèle : verdict global, liste des variables en échec (Event → donnée → grandeur), localisation de la cause, distinction entre problème de *modèle* (contrainte inversée, dates incompatibles) et de *réglage MCMC*, actions chiffrées, contrôle après relance. Les tableaux de phases et d'Events sont produits par l'application avec les valeurs exactes, le modèle les commente.

9 Serveur MCP `seldonh-mcp`

Programme séparé (QtCore seul, `mcp/`) qui expose les 14 outils à un autre assistant (Claude Desktop, Jan, Gemini CLI) par le transport stdio, une ligne JSON-RPC par message : `initialize` (versions 2024-11-05 à 2025-11-25), `tools/list`, `tools/call` (texte + `structuredContent` pour les graphiques), progression du MCMC par `notifications/progress` et annulation. Il relit les réglages de SeldonH (courbes, format des dates, décimales). Différences avec l'application : `analyser_traces` et `tableau` prennent un chemin de fichier (CSV), `creer_densite` écrit le fichier dans un dossier, et la consigne « valeurs déjà affichées » est remplacée par « à reprendre telles quelles », car l'autre assistant ne voit pas la fiche. Il est compilé par les trois scripts d'installation et livré dans le composant `seldonh_app`.

10 Stockage, déploiement et tests

- **Historique** : un fichier JSON par conversation (`turns` pour reprendre le dialogue avec n'importe quel moteur, `display` pour redessiner l'écran), enregistré après la question, à chaque fin de réponse, en cas d'erreur et à la fermeture.

- **Déploiement** : un installeur Qt Installer Framework par système (Windows, Debian/Ubuntu, macOS), avec ou sans modèle (`_noModel`). Windows et Linux : llama.cpp en bibliothèques partagées, 14 variantes CPU + Vulkan chargées dynamiquement ; macOS : statique universel avec Metal.

- **Tests** : `tests/tst_core` (133 tests au 04/11/2026, Linux, Qt 6.4 : compétences, `ChrBuilder` contre des projets réels, méthodes, Sheather–Jones, formats de date, relations (termes mélangés, corrélation), tableaux, mode calculatrice, test d'appel d'outils, serveur MCP, topologie CPU) et `tests/llm_cli` (gabarit, cache KV, points de reprise, troncature, `--threads`, `--cpuinfo`) ; script `bench_cpu.sh` pour le débit CPU.

11 Limites connues

- Les calculs intégrés traitent des dates isolées et un Event sans contrainte ; ils ne remplacent pas ChronoModel pour un modèle complet (phases, contraintes, wiggle matching).

- Codes `process_type` 4, 5, 6 (ID, IF, IDF) de l'archéomagnétisme supposés ; import dans ChronoModel de `.chr` 14C, F14C et density générés à confirmer.
- Appel d'outils (et son test automatique) vérifié contre un serveur factice uniquement ; API Anthropic native non prise en charge ; `seldonh-mcp` sans transport HTTP (ChatGPT, Gemini web) et pas encore essayé dans un vrai assistant ni dans un installeur produit.
- Relations : une datation indépendante mêlée à des traces est supposée hors du modèle ChronoModel ; la position d'une datation dans une question est retrouvée par sa valeur ; la datation `density` n'est lue que dans `relation` (pas encore dans `event` ni `combine`).
- Tableaux : non enregistrés dans l'historique (à joindre de nouveau après réouverture) ; lecteur `.xlsx` fondé sur une API privée de Qt (`QZipReader`) à vérifier avec Qt 6.11.
- Modifications récentes compilées et testées sous Linux (Qt 6.4) ; pas encore recompilées sur le Mac (Qt 6.11).
- Aucun banc d'essai des modèles de langage sur des tâches de datation réelles.